7 - Prática: Mergulho nas IDEs Online para Aprendizado de Máquina (I)

Eduardo Andrei Staudt

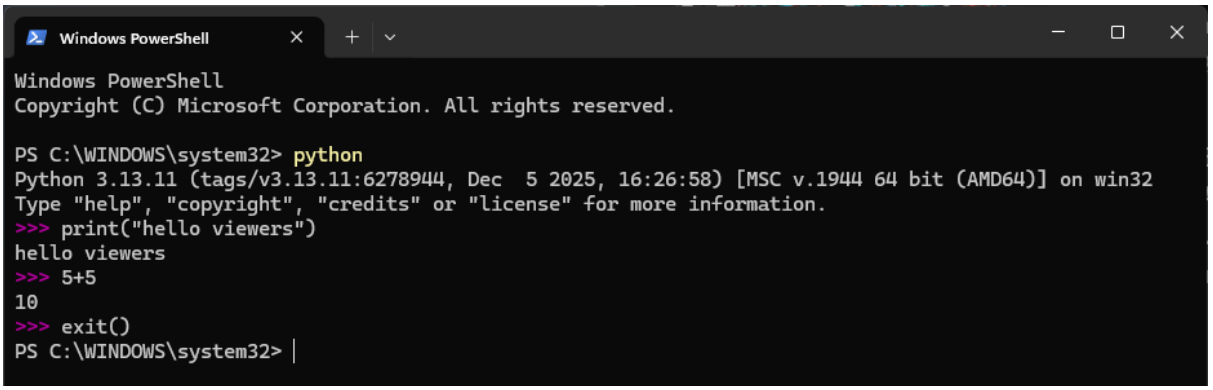
1. Introdução

O objetivo deste card foi aprender a usar as principais IDEs utilizadas em aprendizado de máquina. Para isso, assisti ao vídeo, em que vi as principais e mais utilizadas no mercado, e aprendi em que momento devo usar cada uma e como extrair o potencial delas ao máximo.

2. Desenvolvimento

Comecei assistindo ao vídeo (Rob Mulla). Primeiramente, entendi que iria aprender sobre os diversos tipos de ambientes em que posso trabalhar com Python. O primeiro que vi foi o próprio PowerShell (no meu caso, pois uso Windows), onde basta digitar python nele, como mostrado na imagem abaixo:

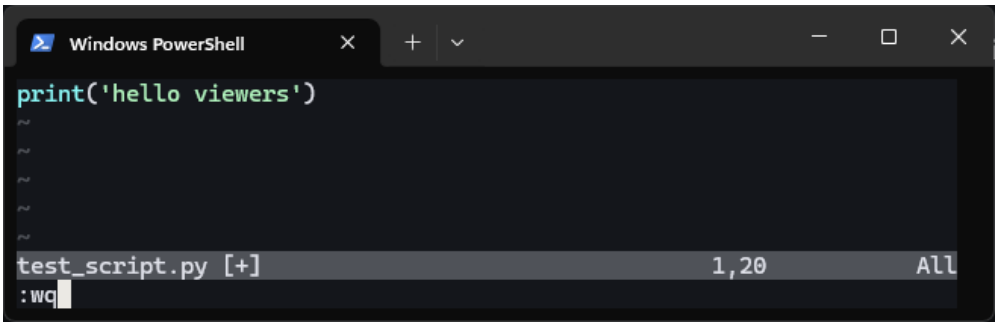
Imagem 1 - Ambiente de Python, em linha de comando.



Fonte: autoria própria, *print* do PowerShell.

Também podemos escrever um *script*, que é um arquivo Python. Para isso, usa-se o comando `nvim nome_script.py`, usei nvim pois não consegui fazer o vim funcionar, e com isso o terminal já abre o arquivo no próprio PowerShell, como mostrado na imagem abaixo:

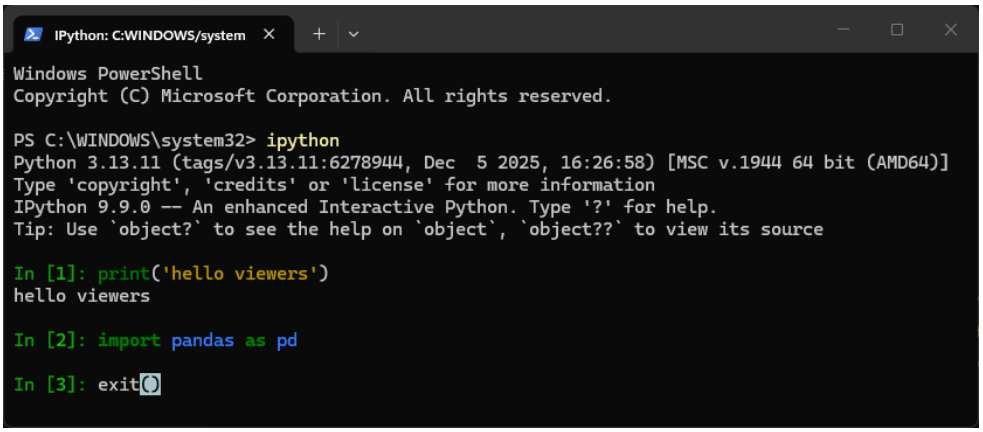
Imagem 2 - Script no PowerShell.



Fonte: autoria própria, *print* do nvim aberto no PowerShell.

Depois disso, foi apresentado o IPython, que também funciona no PowerShell. Ele é mais aprimorado que o terminal comum, pois já tem autocompletar, cores para módulos e, principalmente prompts, que serviram de base para a construção do Jupyter, como mostrado na imagem abaixo:

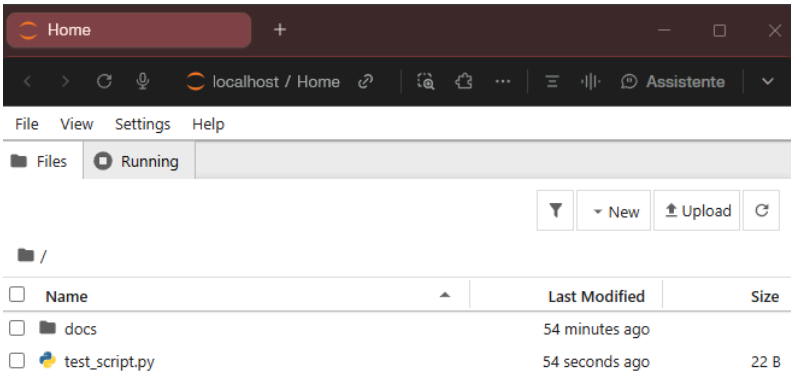
Imagem 3 - Exemplo de uso do IPython.



Fonte: autoria própria, *print* do PowerShell.

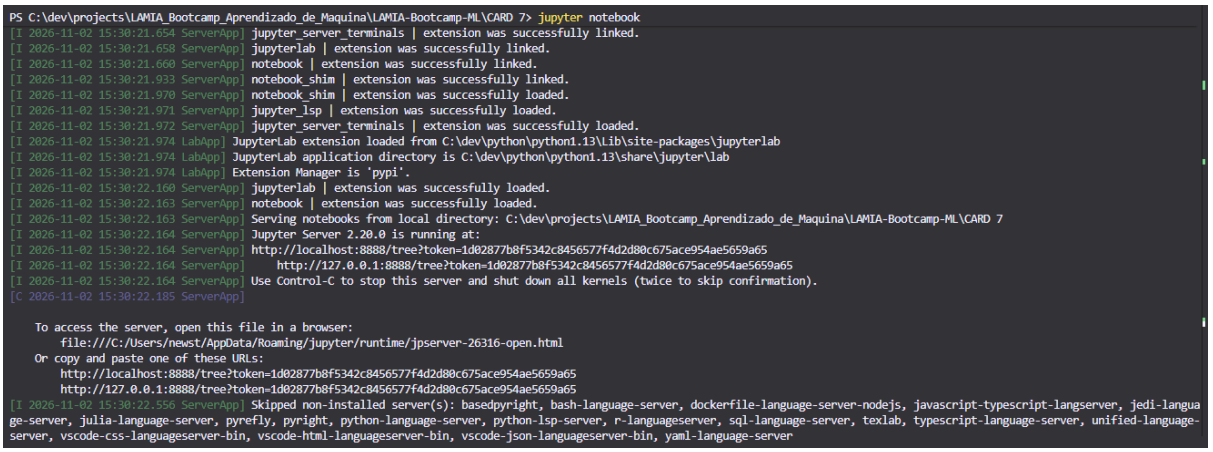
Sabendo que o IPython é a base, aprendi sobre o Jupyter. Para executá-lo, basta rodar o comando jupyter notebook no terminal, na pasta em que você estiver, e ele abre por padrão uma interface web rodando na porta 8888, como mostrado nos exemplos abaixo:

Imagem 4 - Interface do Jupyter.



Fonte: autoria própria, *print* da interface do Jupyter.

Imagem 5 - Iniciando o Jupyter no terminal.



Fonte: autoria própria, *print* do PowerShell.

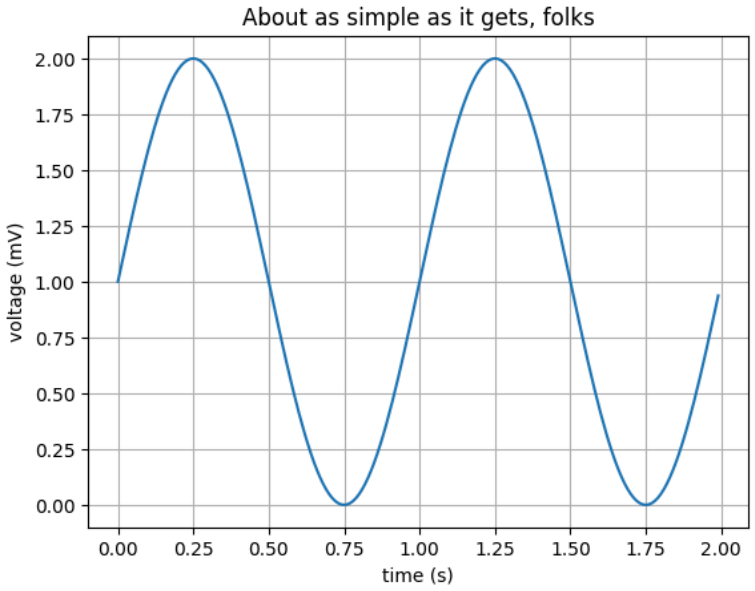
Depois disso, criei o arquivo Notebook_Example, que tem a extensão ipynb, e nele já dá para ver a estrutura de células do IPython. Aprendi as principais coisas que dá para fazer nele, como adicionar, remover e copiar células, navegar por elas, resetar o *kernel* e rodar todas. O kernel é o processo que executa o código das células e guarda na memória as variáveis já criadas. Depois aprendi sobre os dois tipos de células que ele suporta, sendo elas a de Markdown, para escrever textos e explicar o *notebook*, e a de Python, onde se escreve e executa código. Uma das coisas legais dele é que podemos importar, exportar e gerar imagens dentro dele e visualizá-las no próprio *notebook*, como no exemplo abaixo:

Imagem 6 - Exemplo de gráfico gerado dentro do Jupyter.

```
# Data for plotting
t = np.arange(0.0, 2.0, 0.01)
s = 1 + np.sin(2 * np.pi * t)

fig, ax = plt.subplots()
ax.plot(t, s)

ax.set(xlabel='time (s)', ylabel='voltage (mV)', title='About as simple as it gets, folks')
ax.grid()
```

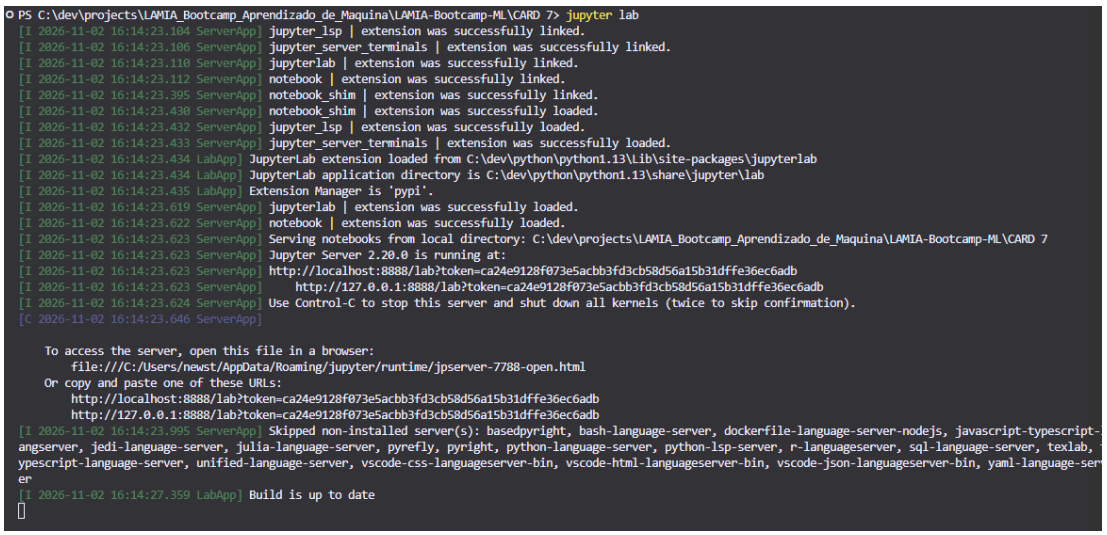


Fonte: autoria própria feita no Jupyter.

O mais legal do Jupyter é poder escrever e explicar todo o seu processo de pensamento, facilitando muito a leitura quando o *notebook* é compartilhado.

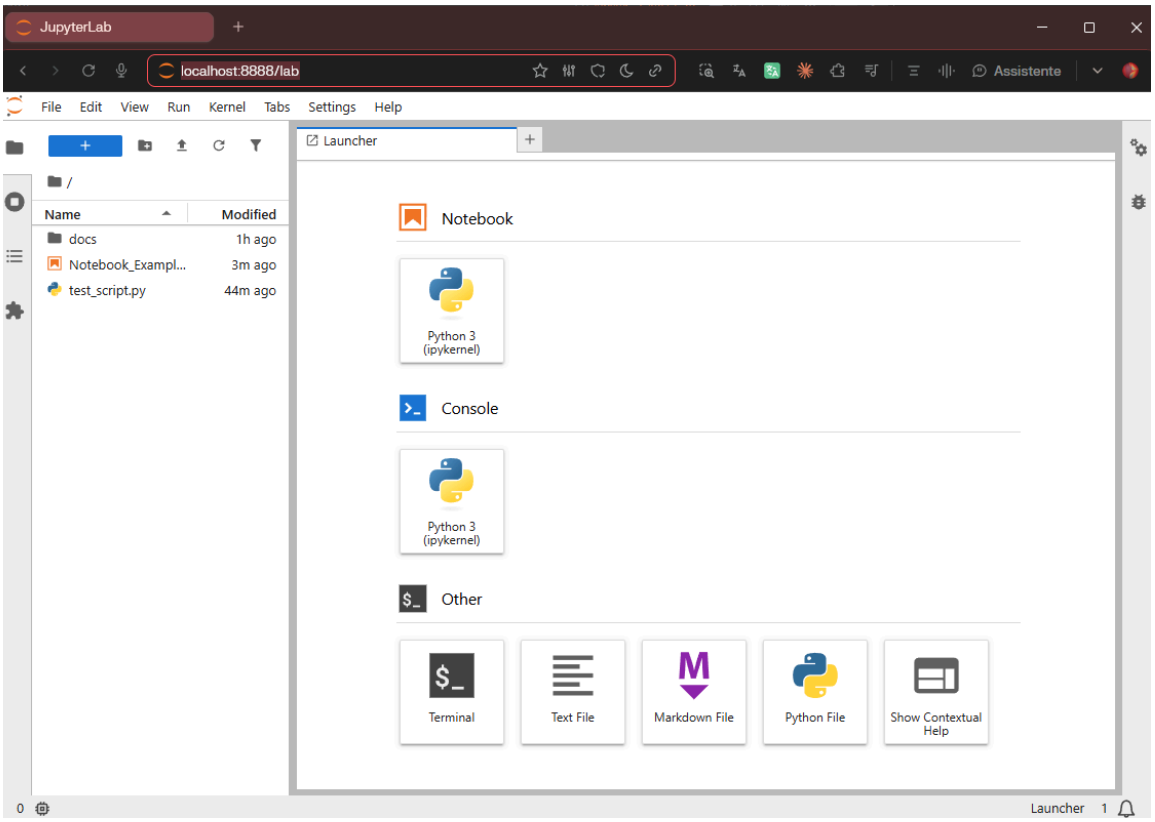
Depois fui apresentado a um Jupyter mais completo, o JupyterLab. Para testá-lo, executei o comando jupyter lab no terminal. O resultado está nas imagens abaixo.

Imagem 7 - Executando o JupyterLab no terminal.



Fonte: autoria própria, *print* do PowerShell.

Imagem 8 - Interface do JupyterLab.

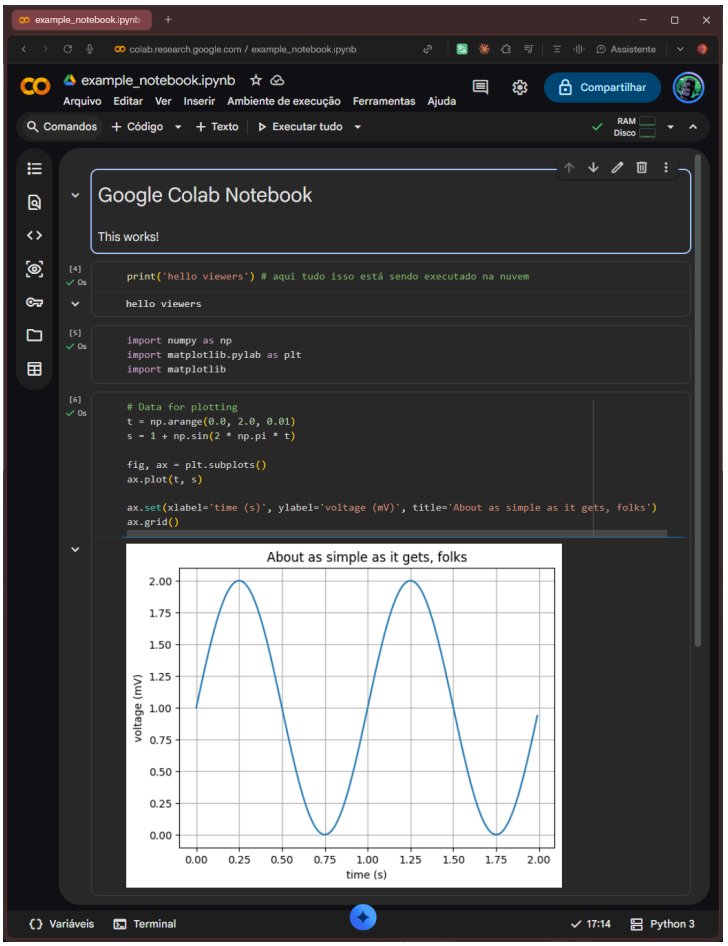


Fonte: autoria própria, *print* do JupyterLab.

Dentro do JupyterLab, aprendi um pouco mais sobre ele. Vi que possui uma seção onde consigo ver todos os *kernels* que estão sendo executados em segundo plano. Tem também um sistema de abas, facilitando a navegação entre *notebooks* e arquivos, um indexador de Markdown e uma aba de extensões, o que abre uma grande gama de possibilidades de personalização. Por fim, ele permite também exportar o *notebook* em mais formatos, facilitando que pessoas que não o tenham instalado também possam utilizá-lo. Além dessas facilidades, ele também herda todas as do Jupyter.

Depois disso, fui apresentado ao Google Colab, que, diferente dos anteriores, é uma plataforma que funciona em nuvem e é utilizada no navegador. Ser em nuvem é muito interessante, pois isso permite que você execute código sem precisar ter tudo instalado localmente. Por exemplo, você pode usar o Python, o NumPy e o Pandas sem precisar baixá-los, apenas importando-os, já que o próprio Colab já os tem instalados. Caso precise baixar alguma versão ou biblioteca que ele não tenha instalada, você pode baixá-la e isso não ocupará espaço no seu computador. Na imagem abaixo, executei os mesmos códigos do *notebook* do Jupyter.

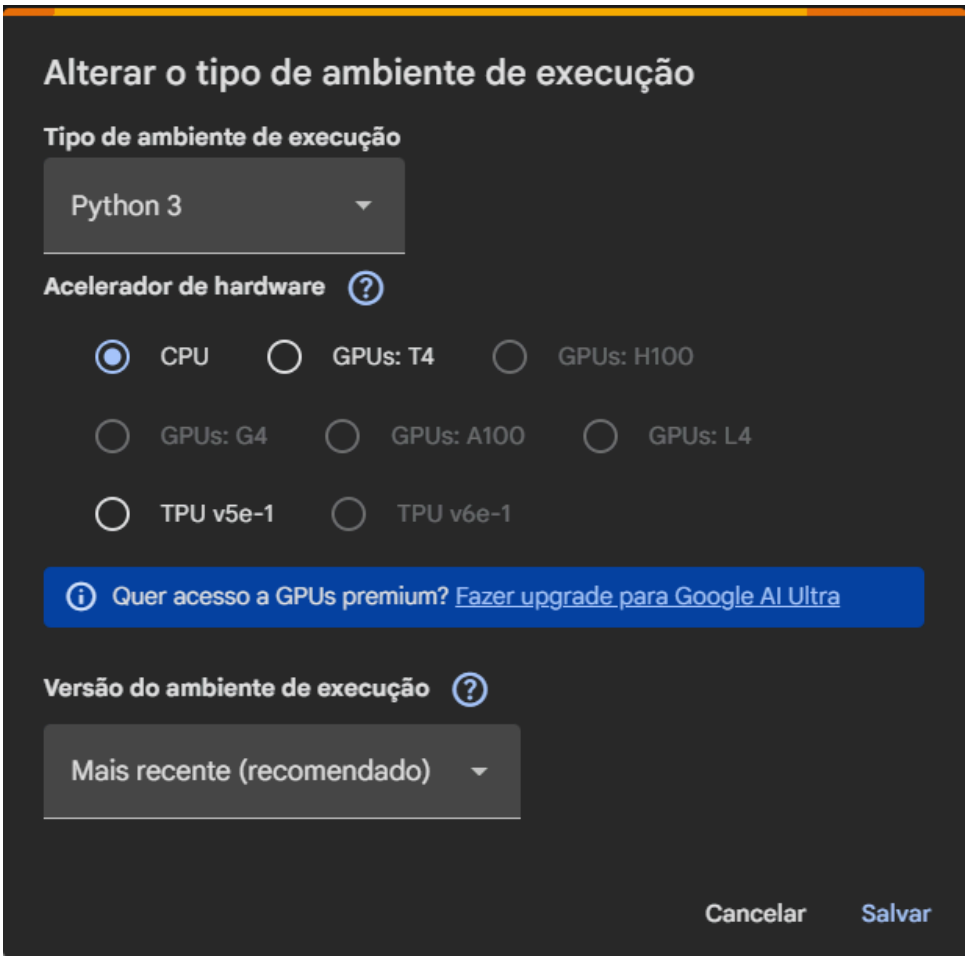
Imagem 9 - Tela inicial da interface do Google Colab.



Fonte: autoria própria, *print* do Google Colab.

O Colab dá acesso a uma quantidade de RAM e de disco. No meu caso, 12,7 GB de RAM e 107,7 GB de disco. Não foi apresentado no vídeo, mas acho interessante comentar a possibilidade de utilizar GPU no *notebook*. A imagem abaixo mostra a tela dos ambientes de execução disponíveis:

Imagem 10 - Ambientes de execução plano gratuito.

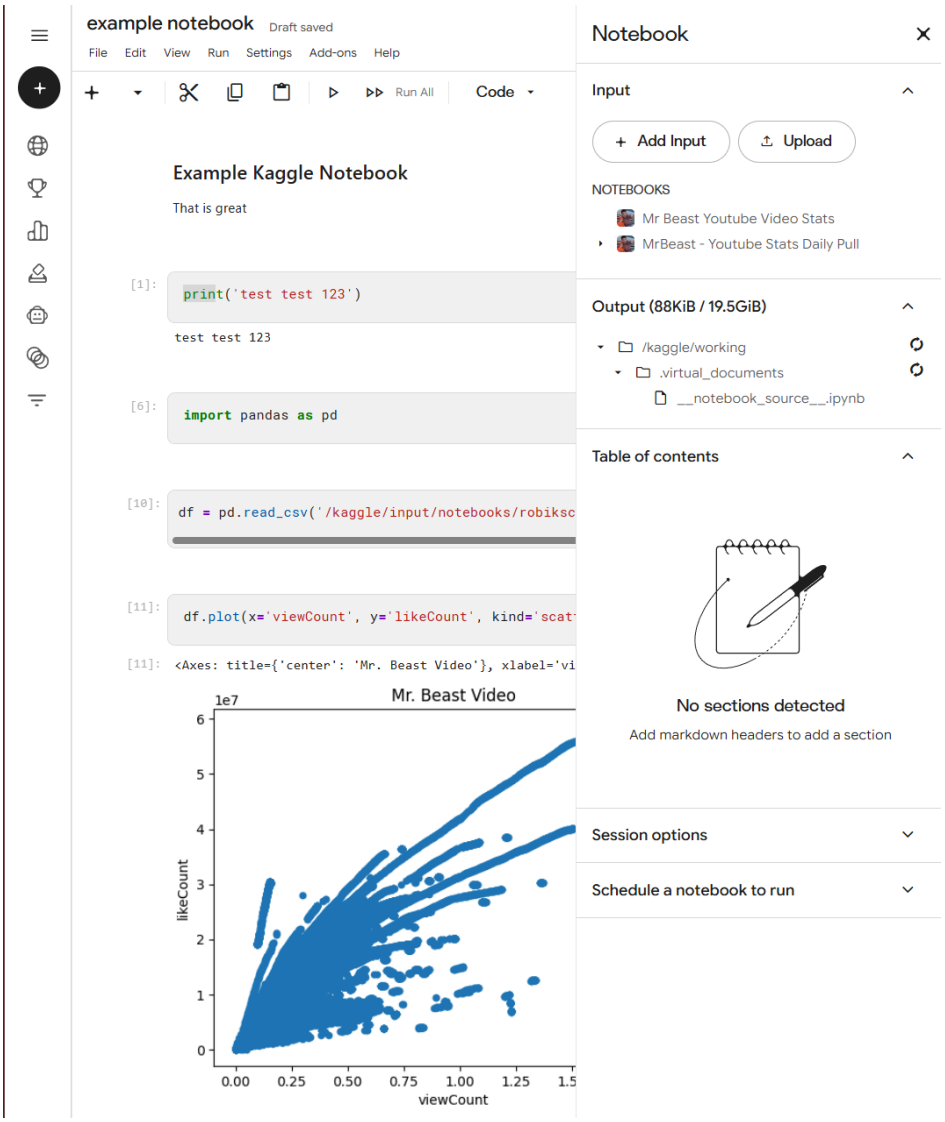


Fonte: autoria própria, *print* do Google Colab.

No plano gratuito, existem estas possibilidades: a CPU, que é o padrão para execução geral sem GPU, a T4 GPU, indicada para treinar redes neurais, e a TPU, voltada a cargas muito grandes e que exige compatibilidade com TensorFlow/JAX.

Depois disso, fui para o último ambiente de desenvolvimento, o Kaggle Notebooks. Ele também é uma plataforma em nuvem utilizada pelo navegador e usa várias das funcionalidades do Jupyter e do Colab em segundo plano, portanto herda muita coisa. Na imagem abaixo, mostro a interface dele, agora utilizando uma das particularidades mais legais, que é poder usar com muita facilidade as bases de dados públicas disponíveis dentro da plataforma. Nesse caso, importei uma base que contém algumas estatísticas do canal do MrBeast.

Imagem 11 - Tela inicial da interface do Kaggle Notebooks.

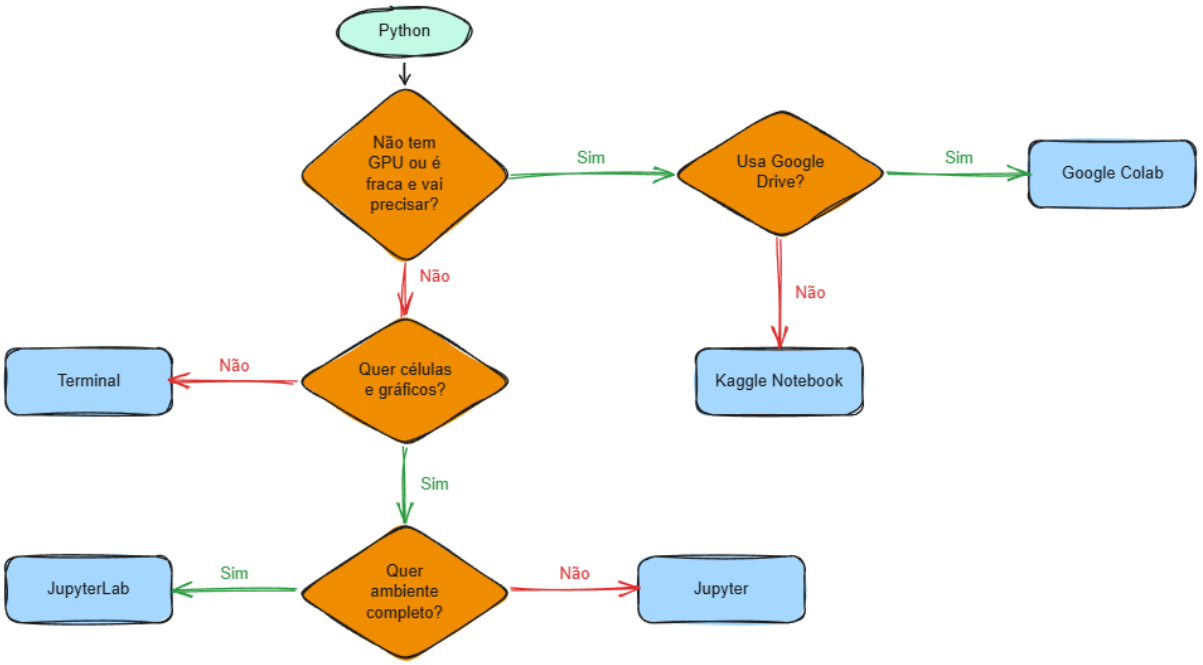


Fonte: autoria própria, *print* do Kaggle Notebook.

Depois aprendi alguns atalhos para facilitar o uso dos *notebooks*. Eles são:

- **Shift + Enter**: rodar a célula atual e ir para a próxima.
- **Ctrl + Shift + Seta cima**: move a célula para cima.
- **Ctrl + Shift + Seta baixo**: move a célula para baixo.
- **Ctrl + Enter**: rodar a célula e permanecer nela.
- **Setas**: movimentação entre células.
- **Esc**: sair do modo de edição da célula.
- **Esc + J**: descer para a célula de baixo.
- **Esc + K**: subir para a célula de cima.
- **Esc + A**: adiciona célula acima.
- **Esc + B**: adiciona célula abaixo.
- **Esc + D, D**: exclui a célula.
- **Esc + Z**: restaura a célula excluída.

Imagem 12 - Fluxograma de uso das principais IDEs.



Fonte: autoria própria feita no Excalidraw.

Por fim, fiz esse fluxograma da Imagem 12 para explicar quando usar cada uma das IDEs. Por exemplo, se você precisa usar uma GPU ou tem uma mais fraca do que a disponibilizada pelas plataformas em nuvem, use o Google Colab se utiliza o Google Drive, ou o Kaggle Notebooks se não utiliza. Se não precisa se preocupar com isso e quer apenas executar código ou rodar na sua GPU ou CPU, use o Jupyter Notebook ou, se quiser um ambiente mais completo, o JupyterLab. Além desses, há o terminal no PowerShell, mas ele é bem limitado, sem facilidades e sem geração de imagens.

3. Conclusão

O vídeo me ensinou a utilizar algumas das ferramentas e IDEs mais utilizadas no aprendizado de máquina, aprendi a escolher a melhor para o meu caso, além de alguns atalhos que podem facilitar o meu trabalho no dia a dia. No caso dos *notebooks* Jupyter, Jupyter Lab, Google Colab e Kaggle Notebooks, eles são ferramentas de extrema importância, principalmente para momentos em que é interessante compartilhar código com outras pessoas e navegar entre células.

4. Referências

MULLA, Rob. **Jupyter Notebook Complete Beginner Guide - From Jupyter to Jupyterlab, Google Colab and Kaggle!** YouTube, 29 jan. 2022. Disponível em: https://www.youtube.com/watch?v=5pf0_bpNbkw. Acesso em: 30 out. 2026.